

Infosys DSA Topics & Concepts Checklist



Timeline: Jan 9-17, 2026 (9 days)

Test Date: January 18, 2026

Target: Get 2/4 problems correct (mix of easy-medium)



1 ARRAYS (Basics + Logic Building)

Core Concepts

- Array declaration, initialization, and traversal
- 1D and 2D arrays
- Array manipulation (insertion, deletion, updating)

Key Problems Patterns

- **Searching:** Linear search, Binary search
- **Sorting:** Bubble, Selection, Insertion, Merge, Quick sort
- **Two Pointer technique**

- **Sliding Window**
- **Prefix Sum / Cumulative Sum**
- **Kadane's Algorithm** (Maximum Subarray)
- Array rotation (left/right)
- Finding duplicates/missing elements
- Merging sorted arrays

Must-Know Problems

- Two Sum
 - Remove Duplicates from Sorted Array
 - Product of Array Except Self
 - Maximum Subarray (Kadane's)
 - Rotate Array
 - Best Time to Buy and Sell Stock
-

2 STRINGS (Basics + Logic Building)

Core Concepts

- String immutability in Java
- StringBuilder vs String
- Character arrays
- String manipulation methods

Key Problem Patterns

- **String reversal**
- **Palindrome checking**
- **Anagram detection**
- **Pattern matching**
- **Substring problems**
- Character frequency counting
- Longest substring without repeating characters
- String compression

Must-Know Problems

- Valid Anagram
 - Group Anagrams
 - Longest Substring Without Repeating Characters
 - Reverse String/Words
 - Valid Palindrome
 - String to Integer (atoi)
-

3 LINKED LISTS (Easy Level Basics + Logic)

Core Concepts

- Node structure (data + next pointer)
- Singly Linked List
- Doubly Linked List
- Circular Linked List
- Traversal, insertion, deletion

Key Problem Patterns

- **Two Pointer / Fast-Slow Pointer (Floyd's Cycle Detection)**
- Reversing linked lists
- Detecting and removing cycles
- Finding middle element
- Merging lists
- Removing nodes

Must-Know Problems

- Reverse Linked List
 - Detect Cycle in Linked List
 - Merge Two Sorted Lists
 - Remove Nth Node From End
 - Find Middle of Linked List
 - Add Two Numbers (as Linked Lists)
 - Intersection of Two Linked Lists
-

STACKS (Basics + Logic Building)

Core Concepts

- LIFO principle
- Push, Pop, Peek operations
- Stack implementation using arrays/LinkedList
- Stack using Java Collections (Stack class)

Key Problem Patterns

- **Balanced parentheses**
- **Next Greater/Smaller Element**
- **Expression evaluation** (postfix, prefix)
- **Monotonic stack**
- Stock span problems
- Histogram problems

Must-Know Problems

- Valid Parentheses

- Implement Stack using Queues
 - Evaluate Reverse Polish Notation
 - Next Greater Element
 - Min Stack
 - Daily Temperatures
-

5 QUEUES (Basics + Logic Building)

Core Concepts

- FIFO principle
- Enqueue, Dequeue operations
- Circular Queue
- Deque (Double-ended queue)
- Priority Queue
- Queue implementation using arrays/LinkedList

Key Problem Patterns

- Queue using two stacks
- Sliding window maximum
- Level order traversal (for trees)
- BFS applications

Must-Know Problems

- Implement Queue using Stacks
 - Sliding Window Maximum
 - First Non-repeating Character in Stream
 - Circular Queue Implementation
 - Generate Binary Numbers
-

6 HASH MAPS / HASHING (Basics + Logic Building)

Core Concepts

- HashMap, HashSet in Java

- Hash function basics
- Collision handling (chaining, open addressing)
- Time complexity: $O(1)$ average for insert/search/delete

Key Problem Patterns

- **Frequency counting**
- **Two Sum variants**
- **Subarray sum problems**
- **Anagram grouping**
- **Longest substring problems**
- Finding duplicates
- LRU Cache design

Must-Know Problems

- Two Sum
- Group Anagrams
- Subarray Sum Equals K
- Longest Substring Without Repeating Characters

- Valid Anagram
 - First Unique Character
 - Top K Frequent Elements
-

7 TREES (Basics + Logic Building)

Core Concepts

- Binary Tree structure (node, left, right)
- Binary Search Tree (BST) properties
- Tree traversals:
 - **Inorder** (Left-Root-Right)
 - **Preorder** (Root-Left-Right)
 - **Postorder** (Left-Right-Root)
 - **Level Order** (BFS)
- Height/Depth of tree
- Balanced trees concept

Key Problem Patterns

- **Recursive traversals**
- **Level order traversal (BFS)**
- **Tree construction** from traversals
- **BST operations** (search, insert, delete)
- **Lowest Common Ancestor (LCA)**
- **Path sum problems**
- **Tree diameter/height**

Must-Know Problems

- **Binary Tree Inorder/Preorder/Postorder Traversal**
- **Level Order Traversal**
- **Maximum Depth of Binary Tree**
- **Same Tree**
- **Validate BST**
- **Lowest Common Ancestor**
- **Kth Smallest Element in BST**

- Construct Tree from Preorder and Inorder
-

8 RECURSION (Medium Analytical)

Core Concepts

- Base case and recursive case
- Call stack understanding
- Recursion tree visualization
- Tail recursion
- Memoization basics

Key Problem Patterns

- **Divide and Conquer**
- Factorial, Fibonacci
- Tower of Hanoi
- Generating combinations/permutations
- Tree recursion

Must-Know Problems

- Fibonacci Number
 - Power(x, n)
 - Reverse a String/Linked List (recursive)
 - Generate Parentheses
 - Subsets
 - Permutations
-

9 BACKTRACKING (Medium Analytical)

Core Concepts

- Decision tree exploration
- Choose → Explore → Unchoose pattern
- Pruning invalid paths
- State space tree

Key Problem Patterns

- **Permutations and Combinations**
- **Subset generation**
- **N-Queens problem**
- **Sudoku solver**
- **Maze problems** (Rat in a Maze)
- Word search in grid

Must-Know Problems

- Subsets
- Permutations
- Combination Sum
- Word Search
- N-Queens
- Rat in a Maze
- Sudoku Solver
- Letter Combinations of a Phone Number

10 GREEDY ALGORITHMS (Medium Analytical)

Core Concepts

- Making locally optimal choices
- Greedy choice property
- Optimal substructure

Key Problem Patterns

- **Activity/Interval Selection**
- **Fractional Knapsack**
- **Job Sequencing**
- **Huffman Coding**
- Coin change (greedy variant)
- Jump game problems

Must-Know Problems

- Jump Game I & II

- Non-overlapping Intervals
 - Assign Cookies
 - Fractional Knapsack
 - Minimum Platforms
 - Activity Selection
 - Maximum Subarray (can use greedy)
-

1 1 OPTIMIZATION-BASED PROBLEMS (Dynamic Programming)

Core Concepts

- Overlapping subproblems
- Optimal substructure
- **Memoization** (Top-Down)
- **Tabulation** (Bottom-Up)
- State definition
- Recurrence relation

Key Problem Patterns

- **0/1 Knapsack**
- **Unbounded Knapsack**
- **Longest Common Subsequence (LCS)**
- **Longest Increasing Subsequence (LIS)**
- **Coin Change**
- **Matrix Chain Multiplication**
- Fibonacci variants
- Grid path problems
- Substring/subsequence problems

Must-Know Problems

- Climbing Stairs
- Fibonacci Number
- Coin Change
- 0/1 Knapsack
- Longest Common Subsequence

- Longest Increasing Subsequence
 - House Robber
 - Edit Distance
 - Minimum Path Sum
-

COMPLEXITY ANALYSIS (Critical!)

Time Complexity

- $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$, $O(n!)$
- Analyzing loops, recursion
- Best/Average/Worst case

Space Complexity

- Auxiliary space
 - Recursion stack space
 - In-place vs extra space
-

JAVA-SPECIFIC ESSENTIALS

Collections Framework

- ArrayList, LinkedList
- Stack, Queue, Deque
- HashMap, HashSet, TreeMap, TreeSet
- PriorityQueue

Important Methods

- Arrays.sort(), Collections.sort()
- String methods: charAt(), substring(), split()
- StringBuilder operations
- Integer.parseInt(), Character.isDigit()

Edge Cases to Handle

- Null checks
- Empty arrays/strings
- Integer overflow (use long when needed)

- Index out of bounds
-

PRACTICE STRATEGY

Daily Structure (9 days)

1. **Morning (1-2 hrs):** Watch videos for the topic
2. **Evening (2-3 hrs):** Solve 5-8 problems (mix easy-medium)
3. **Night:** Review mistakes, optimize solutions

Problem Sources

- **LeetCode** (primary for medium problems)
- **GeeksforGeeks** (good for basics + company-specific)
- **HackerRank** (good for practice)

Mock Test Approach

- Solve 4 problems in 90-120 minutes
- Target: Get 2/4 correct consistently

- Practice in IDE (Eclipse/IntelliJ)
-

FINAL CHECKLIST BEFORE TEST

- ☐ Can implement all basic data structures from scratch
 - ☐ Know time/space complexity of common operations
 - ☐ Comfortable with Java Collections Framework
 - ☐ Can solve 2/4 medium problems correctly
 - ☐ Practiced edge case handling
 - ☐ Done 2-3 full mock tests
 - ☐ Revised weak topics from day 9
-

Good luck! Stay calm, code carefully, and trust your preparation! 🚀